

5.1 Querying Databases

DBMS is for:

- Storing Information
- Organizing Information
- Modifying Information
- Searching for Information

Relational Algebra is an underlying computation framework for answering questions.

Query Languages

- A "computer language" to communicate with a database for asking questions on the information stored within.
- The design of a query language depends on
 - (Physical) Data Model: How is data organized in the database?
 - Query Capabilities: What types of ^{questions} ~~capabilities~~ can be asked?
 - Target users: Who will use the query language?
 - Programmers?
 - Non-programmers?
 - Everyday users?

Query Language: An Example

E.g. search by image in web search.

- Data Model: Collection of images
- Query Capabilities: Finding similar images
- Target users: Everyday users.

5.2 SQL

- Structured Query Language
 - Pronounced "s-q-l" or "sequel".
 - Standard language for querying and manipulating data.
 - Created at IBM, since 1970s.
 - Standardized in 1986, Many revisions were made since then:
 - SQL-86, SQL-89, SQL-92, SQL-1999, ...
SQL:2016.
 - Vendors support various subsets of these.
- ~~also~~
- ### SQL is Declarative: What, Not How!
- SQL is a very-high-level language: It is declarative.
 - Using a consistent set of keyword English templates.
 - You only say what you want, and not how to get it.
 - Unlike in "high-level" programming languages like C++ or Python.
not declarative but procedural

can be both easy to use and efficient

- Ease of use: SQL targets both professional programmer and the more infrequent database users.
- Efficiency
 - Even though programmers are smart, database is dynamic--data can change!
 - declarative queries can be "optimized" by system "just in time".

5.3 Basic Form

SELECT - FROM - WHERE

Basic Form: SELECT - FROM - WHERE

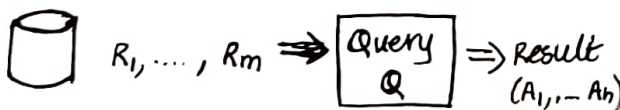
SELECT $A_1, \dots, A_n$ $\leftarrow$ Attributes to return
FROM $R_1, \dots, R_m$ $\leftarrow$ Relations to query
WHERE Condition $\leftarrow$ ^{from} Condition that results must satisfy.

EX.

SELECT name, year FROM students
WHERE major = "ECON"

SQL Query: Relations In, Relations Out

- Input: Relations $R_1, \dots, R_m$ (a subset of relations in DB)
- output: Relation with Attributes $A_1, \dots, A_n$
 - No name -- you can call it the "result" relation



(28)

• Closure Property:

- SQL, like relational algebra is closed with respect to the relational model.
- Each operation takes one or more relations and returns a relation.
- We can thus use a query as a relation.
 - Anywhere when a relation is expected.
 - E.g., the concept of "view".

What does it mean?

- Combine (Cartesian product) the relations in the FROM clause.
- Filter it by the condition in the WHERE clause.
- Output the tuples with the attributes in the SELECT clause.
- In terms of relational algebra:
 $\pi_{A_1, \dots, A_n} (\sigma_c (R_1 \times \dots \times R_m))$

FROM clause

- Specify one or more "source" relations in DB: FROM $R_1, \dots, R_m$
- Tuple Variable: You can use an "alias" to name a relation.
 - For referencing in WHERE and SELECT.
 - Think of it as representing a tuple in the relation.
- Useful for multiple reasons
 - To use as a short name:
 - FROM Favourites F, ...
 - To distinguish multiple instances of the same relation.
 - FROM Drinkers D1, Drinkers D2

Referring to an Attribute

- If no ambiguity, simply use its name
- If ambiguity (or simply for readability)
 - $\langle \text{relation} \rangle . \langle \text{attribute} \rangle$, e.g., Drinkers.name, Bars.name.
 - $\langle \text{tuple variable} \rangle . \langle \text{attribute} \rangle$, e.g., F.beer, D1.name, D2.name.

SELECT Clause

- Specify each attribute: `SELECT A1, ..., An`
- Get all attributes: `SELECT *`
- Rename attributes: `SELECT ..., attr AS new-name`
- Transform attributes by expression:
`SELECT ..., expression AS name, ...`
 - An expression is a combination of one or more values, operators, and SQL functions that evaluates to a value.

WHERE Clause

- WHERE C
- C can be
 - simple condition, or
 - complex condition consisting of multiple simple conditions connected with Boolean operators AND, OR, NOT and parentheses

Conditions in WHERE clause

(29)

- Attributes: from the relations in FROM.
 - prefixed by relation name or tuple variable to uniquely reference.
- Arithmetic operations.
 - `price * discount < 1.0`.
- String operations
 - `UPPER(bar) = 'GREEN BAR'`
- Comparison operators: `=`, `<`, `>`, `<=`, `>=`
- Pattern matching: LIKE operator: ~~bar~~ ~~bar~~
`bar LIKE '% Green %'`
 - `%` = "any string"; `-` = "any character"

5.4 NULL Values

What should this query return?

`SELECT bar, beer`

`FROM sells`

`WHERE price < 5.00 OR price >= 5.00`

vs.

`SELECT bar, price FROM sells;`

Not the same!

NULL Values

- Tuples in SQL relations can have NULL as a value for one or more components.
- Meaning depends on context
- Two common cases:
 - Missing value:
 - E.g. we don't know the exact price of a beer.
 - Inapplicable
 - E.g. the grade of a no-grading class.

Comparing NULL's to Values

- When NULL is compared to any value, the truth value is ~~unknown~~ UNKNOWN.
- Thus, SQL needs to employ 3-valued logic: TRUE, FALSE, UNKNOWN
- But a query only produces a tuple in the answer if its truth value for the WHERE clause is TRUE (not FALSE or UNKNOWN)

3-valued Logic Involving UNKNOWN

- AND: TRUE if both are TRUE.
 - UNKNOWN AND TRUE = UNKNOWN
 - UNKNOWN AND FALSE = FALSE
 - UNKNOWN AND UNKNOWN = UNKNOWN
- OR: TRUE if either is TRUE.
 - UNKNOWN OR TRUE = TRUE.
 - UNKNOWN OR FALSE = UNKNOWN
 - UNKNOWN OR UNKNOWN = UNKNOWN
- NOT: Just flip it.
 - NOT UNKNOWN = UNKNOWN

The Surprise Explained

```
SELECT bar, beer
FROM sells
WHERE price < 5.00 OR price >= 5.00;
```

- For a tuple with price = NULL:
 - price < 5.0 = UNKNOWN.
 - price >= 5.0 = UNKNOWN
 - UNKNOWN OR UNKNOWN = UNKNOWN
 - Thus, this tuple is not in the result.

Testing for NULL

- To test and include/exclude NULL:
 - x IS NULL
 - x IS NOT NULL

```
SELECT bar, beer
FROM sells
```

```
WHERE price < 5.00 OR price >= 5.00 OR
price IS NULL.
```

Subqueries

- A parenthesized SELECT-FROM-WHERE contained in a query.
- can contain subqueries.
- can return a relation or a single constant value.
- can be used in any place where a relation or constant values are expected - FROM, WHERE, and even SELECT.

Subqueries that Return Single Constants

- If a subquery is guaranteed to produce one tuple with one attribute, then it can be used as a scalar value.
 - Note it is still a relation, just a special one.
- can be used anywhere a scalar value is expected.
 - WHERE -- our focus.
 - SELECT too!
- "single constraint" can be guaranteed by:
 - Key constraints
 - E.g., Enrolls (name, number, grade): one grade for a given name and number.
 - Aggregate functions
 - E.g., Avg(price) of all sells tuples.

Subqueries that Return Relations

31

- In general, a subquery (like any query) would return a relation.
- Can be used anywhere a relation is expected.
 - WHERE -- Our focus.
 - FROM too!

Using Relation-Subqueries in WHERE

Conditions.

- Relation subquery returns a relation-- i.e., a set of tuples.
- How to use a relation in a condition?
- We need operators that deal with sets.

Operators that deal with a Relation

a set of tuples

IN (or NOT IN)

- $\langle \text{tuple} \rangle \text{ IN } \langle \text{relation} \rangle$: if the tuple is a member of the relation.

EXISTS (or NOT EXISTS)

- $\text{EXISTS } \langle \text{relation} \rangle$: if the $\langle \text{relation} \rangle$ is not empty.

ANY

- $X \langle \text{OP} \rangle \text{ ANY } \langle \text{relation} \rangle$: $X \text{ OP } (=, <, \text{etc.})$ at least one tuple in the relation.
- $\langle \text{relation} \rangle$ has only one attribute (so each tuple is a single value).

ALL

- $X \langle \text{OP} \rangle \text{ ALL } \langle \text{relation} \rangle$: $X \text{ OP } (=, <, \text{etc.})$ all tuples in the relation.